

```

/* =====
CONFIG.JS - LA ZONE QUE TU PERSONNALISES
=====

Tout ce dont tu as besoin pour brancher TON assistant IA est ici.
Le reste du code (app.js, index.html, style.css) n'a normalement
pas besoin d'être touché - il lit simplement cet objet CONFIG.

Tu peux relancer ce fichier à volonté, autant de fois que tu veux.
===== */

const CONFIG = {

  /* -----
  1. IDENTITÉ DE TON ASSISTANT
  ----- */
  assistant: {
    nom: "Mon Assistant",
    avatarEmoji: "🤖",
    messageAccueil: "Salut ! Je suis ton assistant. Pose-moi une question, ou dem
    // Personnalité / instructions système envoyées au modèle à CHAQUE conversati
    // Utilise des backticks (`) au lieu de guillemets pour écrire sur plusieurs
    // sans limite pratique - écris ici tout ce que tu veux : ton, règles, connai
    // ce qu'il doit faire ou éviter, exemples, etc.
    promptSysteme: `
Tu es un assistant utile, clair et amical. Réponds en français par défaut.

Ici tu peux ajouter, par exemple :
- des règles de ton (ex : "reste toujours concis", "utilise de l'humour léger")
- des connaissances propres à ton usage (ex : infos sur ton projet, ton entrepris
- des instructions de comportement (ex : "pose une question si la demande est vag
- des exemples de réponses que tu aimes

Ajoute autant de lignes que nécessaire - remplace tout ce texte par tes propres i
    `.trim()
  },

  /* -----
  2. MOTEUR DE CHAT (texte)
  -----
  mode:
    "demo"      -> réponses simulées, fonctionne sans rien configurer,
                  utile pour tester l'interface tout de suite.
    "api"       -> appelle une vraie API compatible (OpenAI, Anthropic,
                  Mistral, un modèle open source auto-hébergé type

```

Ollama/LM Studio, etc.)

⚠ Pour le mode "api" : la plupart des API (OpenAI, Anthropic...) bloquent les appels directs depuis un navigateur (CORS) et exigent que la clé secrète ne soit jamais exposée côté client. La bonne pratique : héberger un petit serveur relais (proxy) qui garde ta clé secrète et que ton app appelle à sa place. Le endpoint ci-dessous doit pointer vers CE relais, pas directement vers OpenAI/Anthropic.

```
----- */
chat: {
  mode: "api", // ✅ actif – nécessite que ton relais (voir dossier relais-serv

  api: {
    // ⚠ REMPLACE cette ligne par l'URL Render que tu obtiendras après déploie
    // ex : "https://mon-assistant-relais.onrender.com/api/chat"
    endpoint: "https://REPLACE-PAR-TON-URL-RENDER.onrender.com/api/chat",
    modele: "gpt-4o-mini", // modèle OpenAI utilisé par le relais
    // Aucune clé API ici ! Elle vit uniquement sur le serveur relais (variable
  }
},
```

```
/* -----
3. GÉNÉRATION D'IMAGES
-----

Branché par défaut sur Pollinations.ai : un service gratuit,
open source, et SANS clé API. Il suffit de construire une URL
et de l'utiliser comme source d'image – donc ça marche
immédiatement, sans backend, sans coût.
Doc : https://pollinations.ai
```

```
----- */
image: {
  mode: "pollinations", // "pollinations" (gratuit, prêt à l'emploi) ou "api" (

  pollinations: {
    construireUrl: (prompt) =>
      `https://image.pollinations.ai/prompt/${encodeURIComponent(prompt)}?width
  },

  api: {
    // Si un jour tu préfères Stable Diffusion, DALL·E, etc. via ton relais
    endpoint: "http://localhost:3000/api/image"
  }
},
```

```
/* -----
4. EMOJIS
```

```

-----
Liste affichée dans le sélecteur rapide. Ajoute/retire ce
que tu veux – aucune API requise, ce sont des emojis natifs.
----- */

emojis: [
  "😊", "😂", "😍", "😏", "😎", "😓", "😡", "👍", "👎", "🙏",
  "🎉", "🔥", "💡", "✅", "❌", "❤️", "🚀", "🎨", "🖼️", "🤖",
  "📌", "⭐", "👁️", "💬", "🕒", "📎", "🧠", "🔧", "📷", "🎵"
],

/* -----
5. STOCKAGE
-----

"memoire"  -> les messages disparaissent si on ferme la page
              (par défaut, aucune config requise)
"storage"  -> persistant via window.storage (si tu déploies
              dans un environnement qui le supporte)
----- */

stockage: {
  mode: "memoire"
}

};

```